

# Unitree Week 31

Presentation by Mario Ortiz

# This Week

- Completed Sim2Sim
- Sim2Sim Preparation for Hardware Deployment

# Keyboard Control

- Constructors allow us to implement the keyboard without changing the policy or deploy.yaml.
  - The keyboard will emulate the same UnitreeJoystick object that the existing FSM and velocity observation already have.
- Will set it to Sim2Sim only using this command “./r1\_ctrl --network=lo”
-

# R1 Scene File

- Corrected wrist contact-exclusion names from *wrist\_pitch\_link* to *wrist\_roll\_link*
- The scene uses 24 torque <motor> actuators
  - Replaced the MJLab position-PD actuators as it didn't match the Unitree bridge's PD torque calculations.
- Sensor Array was constructed → in conclusion the scene compiles with 24 actuators and 88 sensor-data values.

# R1 DDS Motor Mapping

In simulate/src/unitree\_sdk2\_bridge.h:

- We map MuJoCo actuator indices to the correct R1 DDS slot

```
static constexpr int r1_map[24] = {  
    0, 1, 2, 3,  
    6, 7, 8, 9,  
    4, 5, 10, 11,  
    12, 13,  
    15, 16,  
    22, 23,  
    17, 18, 19,  
    24, 25, 26  
};
```

In deploy/include/FSM/State\_RLBase.h:

- We fix the RL's state gain assignment → we mapped the policy joint order to DDS motor IDs when assigning gains

```
const int motor_id =  
  
static_cast<int>(env->robot->data.joint_ids_map[i]);  
  
lowcmd->msg_.motor_cmd()[motor_id].kp() =  
    env->robot->data.joint_stiffness[i];  
  
lowcmd->msg_.motor_cmd()[motor_id].kd() =  
    env->robot->data.joint_damping[i];
```

# Keyboard Control

- The simulator and controller by now communicated correctly, but the controller had no usable joystick device inside WSL (The stock FSM expected controller combinations such as “LT + Up” = FixStand, “RT + A” = Velocity, “LT + B” = Passive).
  - Therefore we setup the keyboard to initiate these commands

# Keyboard Control

- In “deploy/robots/r1/main.cpp”, we added

```
if (vm["network"].as<std::string>() == "lo")
{
    FSMState::keyboard = std::make_shared<Keyboard>();

    auto & js = FSMState::lowstate->joystick;

    js.LT.smooth = 1.0f;
    js.RT.smooth = 1.0f;
    js.lx.smooth = 1.0f;
    js.ly.smooth = 1.0f;
    js.rx.smooth = 1.0f;
    js.ry.smooth = 1.0f;

    std::cout << "Keyboard control enabled for Sim2Sim.\n";
    std::cout << "  2 = FixStand | 3 = Velocity | 1 = Passive\n";
    std::cout << "  W/S = forward/back | A/D = left/right\n";
    std::cout << "  Q/E = turn | X or Space = stop\n";
}
```

- This instantiated the existing Keyboard class when running on the loopback interface.
- Smoothing implementation: Added because the commands “LT” and “RT” are smoothed analog trigger values, while “Up.on\_pressed” is a one-time button edge.
  - Therefore we made the trigger values respond immediately, allowing us to press 1,2, or 3 to initiate the next state.

# Emulating the joystick

In “deploy/include/FSM/FSMState.h”

- We expand the original function (which only updated “LowState” and the keyboard (if present) ), so that the keyboard input writes into the same joystick object the exisitic FSM and velocity observation use.
- This allows us to not make a separate keyboard policy observation, and instead just supply virtual operator inputs to the joystick FSM.
-

# Virtual Suspension

- Now that keyboard transitions work, we can use these commands to transition the robot from Passive → FixStand → Velocity and hence get the robot to initiate motion.
- The initial problem: the robot initially collapsed in Passive mode.
  - After further checking, the main issue was the Passive sets position stiffness was set to zero. ( motor.kp() = 0; )

```
cd ~/unitree_rl_mjlab
```

```
sed -i \
```

```
's/^enable_elastic_band:.*\/enable_elastic_band: 1 # Virtual  
suspension for humanoid initialization/' \  
simulate/config.yaml
```

# Virtual Suspension

Resulting Settings:

robot: "r1"

robot\_scene: "src/assets/robots/unitree\_r1/xmls/scene\_r1.xml"

domain\_id: 0

interface: "lo"

use\_joystick: 0

enable\_elastic\_band: 1

This allowed us to initialize the R1 and enter FixStand without the robot falling.  
Provides the scenario to provide the R1 the ability to transition to motion capabilities.

# Build Commands

- Important to specify the build commands

- For the keyboard and gain,

```
cd ~/unitree_rl_mjlab/deploy/robots/r1/build  
make -j4
```

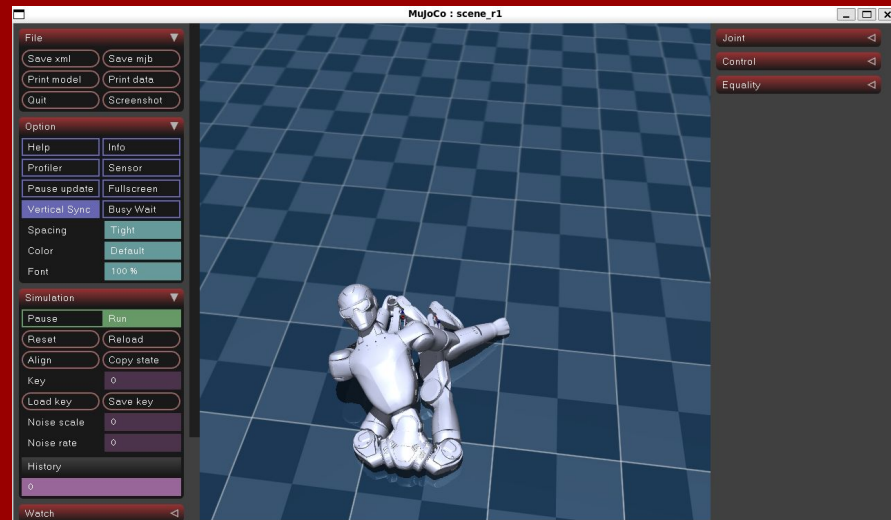
- For the Simular Bridge:

```
cd ~/unitree_rl_mjlab/simulate/build  
make -j4
```

# First Test

Following this protocol: FixStand → Lower Feet to Floor → Velocity → Release Band

- The robot immediately fell once releasing the virtual suspension.
- This signified issues with the RLPolicy



# First Test

- After debugging, it was found that the issue relating to the RL Policy was an action-scale mismatch.
  - It was found that the stock deployment YAML had all the action-scale values to 0.25.
  - As this doesn't align with the scale of the values from the actual training, this needed to be fixed.

In “deploy/robots/r1/config/policy/velocity/v0/params/deploy.yaml”, we change these values to:

actions:

JointPositionAction:

clip: null

joint\_names: [.\*]

scale: [0.15, 0.15, 0.15, 0.15, 0.3125, 0.3125,  
0.15, 0.15, 0.15, 0.15, 0.3125, 0.3125,  
0.15, 0.15,  
0.375, 0.375, 0.4125, 0.4125, 0.4125,  
0.375, 0.375, 0.4125, 0.4125, 0.4125]

File

Save xml

Save mjb

Print model

Print data

Quit

Screenshot

Option

Help

Info

Profiler

Sensor

Pause update

Fullscreen

Vertical Sync

Busy Wait

Spacing

Tight

Color

Default

Font

100 %

Simulation

Pause

Run

Reset

Reload

Align

Copy state

Key

0

Load key

Save key

Noise scale

0

Noise rate

0

History

0

Watch

Physics



Joint

Control

Equality

# Second Test (Successful)

- Detail of test setup:

- **Terminal 1: MuJoCo Simulator**  
(Opens the simulator and outputs actuators/sensor joints; opens MuJoCo, and in MuJoCo is where you can elevate or lower the elastic band which elevates the Robot)

```
conda activate unitree_rl_mjlab
cd ~/unitree_rl_mjlab
MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA \
./simulate/build/unitree_mujoco
```

- Detail of test setup:

- **Terminal 2: R1 Deployment Controller** (Must be in this terminal to initiate commands such as: 1,2,3,w)

```
conda activate unitree_rl_mjlab
cd
~/unitree_rl_mjlab/deploy/robots/r1/build
./r1_ctrl --network=lo
```

# Second Test (Successful)

- ❑ **Step 1: Start both programs**
- ❑ **Step 2: Press 2 in Terminal 2 (Enter FixStand state and wait for the interpolation to finish)**
- ❑ **Step 3: Lower the robot (Focus MuJoCo and press 8 until both feet contact the floor)**
- ❑ **Step 4: Press 3 in Terminal 2 (Enter Velocity state with zero commanded motion)**
- ❑ **Step 5: Release the Band (Focus MuJoCO and press 9. Confirm the R1 stands on its own).**
- ❑ **Step 6: Press W in Terminal 2 (The policy receives the forward command and the robot walks).**

# Next Step:

- Now we have the corrected R1 scene, simulator bridge mapping, RL gain mapping, keyboard emulation, elastic-band configuration, ONNX policy, and corrected deployment YAML.
- Next steps involve changing “--network=lo” to an Ethernet interface.
  - Before this, we set up a controlled audit of the exact R1 EDU hardware configuration, motor ordering, gains, action and observation equivalence, startup mode, communication timing, and a real emergency-stop/dead-man procedure.

# Confirmation

| Joint                      | Training Kp | Training Kd | Effort Limit (N*m)q | Action Scale |
|----------------------------|-------------|-------------|---------------------|--------------|
| Hip, Knee                  | 100         | 2           | 60                  | 0.1500       |
| Ankle Pitch/Roll           | 40          | 2           | 50                  | 0.3125       |
| Waist                      | 100         | 2           | 60                  | 0.1500       |
| Shoulder Pitch/Roll        | 40          | 2           | 60                  | 0.3750       |
| Shoulder Yaw, Elbow, Wrist | 20          | 1           | 33                  | 0.4125       |

# Confirmation

- Policy Timing: 50 Hz  $\rightarrow$  shown from MuJoCo timestep of 0.005s + a decimation of 4.
- Actor observation normalization is enabled  $\rightarrow$  Shown from “obs\_normalization: true” for the actor and uses hidden layers of 512, 246, and 128 neurons with ELU activation.
- Observation Noise is enabled during training  $\rightarrow$  this provides additive noise for angular velocity, project gravity, joint position, and joint velocity.
-

# Confirmation Checklist

- Original Sim2Sim Baseline Works (functional walking policy on MuJoCo)
- Ensured trained policy and deployment config
- Fixed and verified the gait-phase observation (replaced phase implementation that had a counter-like behavior that caused the first policy to phase to be offset to a stateless phase calculator based on  $\text{episode\_length} * \text{step\_dt}$ ).
- Lifecycle/Thread-safety verification (examines the Unitree R1 controller source, SDK wrappers, FSM, publisher/subscriber, action manager, and policy thread).
  - Contained several issues that would affect physical deployment (such as lack of synchronization between the policy worker and FSM)
- Tested a standalone lifecycle model to test first-entry pending behavior, stale-generation rejection, dimension checks, nonfinite-action rejection, result expiry, re-entry behavior, and concurrent worker operation.

# Confirmation Checklist 2

- Designed the LowCmd FSM transaction model.
  - FSM Scheduler now owns a short LowCmd model
- Added a policy input boundary, which rejects any incorrect data (joint positions + velocity, IMU gyro, quaternion, x/y/yaw, etc.)
- Tested the handoff between the real policy and the “worker”
  - ObservationManager, ActionManager, observation functions, ONNX policy, process-action formula,...

# Goals for successful deployment

1. Make the controller safe to operate
2. Confirm the exact R1 EDU Hardware
3. Read-only integration with the physical robot
4. Support actuator and standing tests
5. First Physical RL walking

# Setup Commands for Working Controller

Terminal 1:

```
cd ~/unitree_rl_mjlab  
export MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA  
./simulate/build/unitree_mujoco
```

Terminal 2:

```
cd ~/rl_validation/rl_integration_v2  
./sim_review_build_v1/rl_ctrl_sim_review \  
--sim-review --network=lo
```

# References

- [https://github.com/unitreerobotics/unitree\\_rl\\_mjlab](https://github.com/unitreerobotics/unitree_rl_mjlab)
- [https://github.com/unitreerobotics/unitree\\_rl\\_mjlab/blob/main/doc/setup\\_en.md](https://github.com/unitreerobotics/unitree_rl_mjlab/blob/main/doc/setup_en.md)